

CorrTrack — candidate selection: v1 degenerates into brute force, v2 does not

— **v1** L2 radius cell_size — only filter

share of pairs passing each stage · dense data

Sketch



Selection



no intermediate filter · freq_threshold=0 (full_vector)

Pearson validation



$$\|a-b\|^2 \approx 2(1-r) \Rightarrow r \gtrsim 1 - \tau^2/2$$

cell_size	τ	keeps $r \geq$
1	0.78	0.70 ✓
2	1.55	-0.2 (almost all)
3	2.32	all ($r \geq -1$)

Effect — candidates $\approx$ all pairs $\Rightarrow$ exhaustive validation = **brute force in sketch space** (slower than BF). The sweep scans $\text{cell_size} \in [1, 2, 3]$.

normalized RP

$$\|a-b\| \leq \tau$$

exact, $O(n^2)$

— **v2** *bounded* cosine gate before Pearson

share of pairs passing each stage · dense data

Sketch



sketch_method (5 variants) — drives separability (AUC)

Index



Cosine gate



bounded $[-1, 1]$ · always applied (validate_sketches)

Pearson validation



$\cos(a, b) \approx r$ **directly** — bounded threshold $[-1, 1]$.

- ✓ Gate sketch_threshold cuts **before** the costly Pearson
- ✓ Loose index $\rightarrow$ inflates the cosine (cheap), **not** Pearson
- ✓ Rest = data separability $\rightarrow$ diagnose_select

Effect — set reduced before validation $\Rightarrow$ **Pearson protected.**

RP / fft / median

query_radius (cdist)

$\cos \geq 0.7$

$r \geq 0.8$

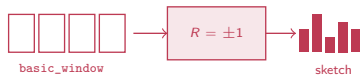
KEY POINT

The **v1 mechanism** (unbounded radius, no filter before Pearson) is **absent in v2**. The “almost everything passes” symptom may return in **v2**, but **bounded by the cosine gate** (Pearson protected) and **driven by data separability**.

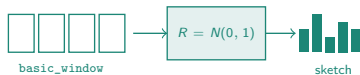
Step 1 — Sketch: projecting the window · v1 vs v2

Role — turn each window into a short vector ($n_vectors$) that **preserves correlation** (Johnson–Lindenstrauss projection).

— **v1** random projection, 1 method



— **v2** v1 port + 4 more methods



- **Rademacher** matrix $\pm 1/\sqrt{n_vectors}$ + toggleVector (order)
- incremental via basicDots
- normalization $z \mid \text{mean_12}$ · Cython kernel

- Gaussian $N(0, 1)$ + znormalize
- **5 methods**: RP, fft_lowpass/topk, median_blocks/phase
- project is **backend-swappable**

KEY DIFFERENCE the v2 sketch is **generalized** (incl. lag-invariant FFT) and **swappable** → it becomes a **separability lever**, not a constant.

Step 2 — Index / neighbor retrieval · v1 vs v2

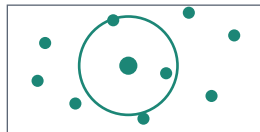
Role — for each window, find its neighbors in sketch space (**raw candidates**).

— **v1** 1 structure, 1D key



- bptree (sorted list) or flat
- range $[v_0 \pm \tau]$ **then** full-L2 refine (`find_pairs_full`)
- `neg_corr` → also inserts `-vec`

— **v2** 13 indexes, isolated retrieval



- **13 indexes:** grid / bst / kdtree / hnsw / annoy ...
- `query_batch` = **BLAS cdist** (tree off in batch)
- `query_radius` · grid = multi-table LSH

KEY DIFFERENCE

v1 = one 1D structure *welded* to selection; v2 = 13 **interchangeable** indexes, retrieval **isolated**.

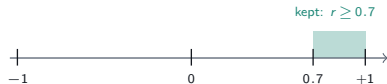
Step 3 — Selection (v1) $\equiv$ Index+Gate (v2): the bridge

Role — decide which pairs deserve exact validation. Same r -axis below: what each version keeps.

— v1 euclidean radius, unbounded



— v2 cosine, bounded



- filter = cell_size (τ): $r \gtrsim 1 - \tau^2/2$
- vote freq_threshold; = 0 in full_vector (default)
- 1 knob, couples retrieval/filter · $\tau > 2 \Rightarrow$ keeps all

- retrieval query_radius separate from filter
- gate sketch_threshold: $\cos \approx r$, **bounded** $[-1, 1]$
- neg_corr $\rightarrow |\cos|$

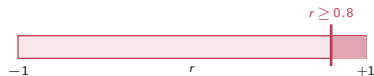
same r (JL) — setting sketch_threshold = $1 - \tau^2/2$ reproduces the v1 filter *exactly*

KEY DIFFERENCE v2 **decouples** retrieval and filtering *and* bounds the threshold $\rightarrow$ v1's “keep everything” becomes inexpressible.

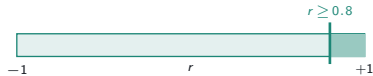
Step 4 — Exact Pearson validation · v1 vs v2

Role — **exact** correlation on raw windows: confirm or reject each candidate.

— **v1** Pearson + guards



— **v2** batch Pearson + backend



- `fast_corr_and_dist` (+ `min_dist`)
- rejects: $\text{std} < 10^{-3}$ and **spiked** ($\text{kurtosis} > 5$)
- Cython batch, threads

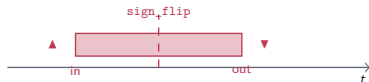
- `backend.correlate` per batch (swappable)
- reject: `std_threshold` only (**no** kurtosis)
- returns the **signed lag** ($t_A - t_B$)

KEY DIFFERENCE same Pearson; v1 adds an anti-spike (kurtosis) + `min_dist`; v2 = `std_threshold` only, swappable backend, native signed lag.

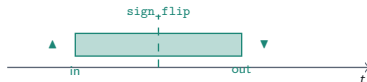
Step 5 — Persistence monitoring · v1 vs v2

Role — track over time the **lifetime** of correlations and their **anomalies** (onset, drop, sign change).

— **v1** coupled to compute + artifacts



— **v2** pure bookkeeping



- `corr_lengths` / `corr_anomalies`, key (id_1, id_2, lag)
- `_in_corr` + `_out_corr`, Cython kernel
- coupled to artifact writing (disk spill)

- Monitor class: active / episodes
- typed anomalies: in / out / sign_flip
- **no backend dependency**

KEY DIFFERENCE same model (lifetime + anomalies); v2 is **decoupled** from compute and artifacts, where v1 mixes Cython kernel and disk spill.